Relational Latent World Learning

An Interaction-Centred Research Position on Effective Variables, Task Constraints, and Action

This paper describes a research plan, not finished results. Its one big question is: can a machine (or an agent) figure out how a part of the world works simply by trying things and watching what happens*, and then* reuse *that understanding both to keep learning and to actually get things done? The words are kept simple below, and every abstract idea is followed by a concrete example from embodied robotics, where an agent observes a scene, acts through a body, and learns from the resulting physical change.*

How to read this paper (a short glossary)

Throughout the paper we use a small set of technical words. Here they are in plain language, each with an embodied-robotics example.

Term	Plain meaning	Embodied-robotics example
Agent	Something that can see and act in a situation	A mobile manipulator
Observation O_t	What can be sensed at one moment	RGB-D images and joint readings
Action a_t	What is physically executed	Moving a gripper and closing its fingers
State S_t	A short useful summary of the current situation	Cup pose, drawer position, and contact state
Transformation (TP) p	*What actually changed*, not the action name	"The cup slid left" rather than "push"
Mechanism Z_D	A reusable cause–effect unit that recurs	Contact force producing object motion
Relation R_D	A one-way effect of one mechanism on another	A fixture constraining the motion caused by a push
World W_D	The learned model of how the domain works	Reusable manipulation mechanics of a workspace
Task C_T	A goal or desired outcome	Place the cup inside the tray
Exploration	Choosing the next action to reduce confusion	Probing an object to test whether it is fixed
Identifiability	Whether data distinguish explanations	Testing whether motion came from pushing or support movement

The key move of the whole paper is this: the agent is allowed to describe what happened in terms of reusable mechanisms *and* relations between them*, and is only allowed to keep a mechanism or a relation if it keeps being useful on new situations. A description is not accepted just because it looks pretty — it must* earn its place *by predicting things that were never shown to it.*

Abstract

This paper asks whether an agent that can sense and act can discover a compact, relational account of a domain from interaction alone, and then reuse that account for two jobs at once: exploring on its own, and completing tasks. The proposed answer is neither a complete hand-written physics engine nor a black-box task robot. It is a stack of *useful summaries*, each sitting on top of the last.

In plain terms, the stack is built like this:

- A *persistent state* S_t makes "change" measurable, so we can compare one moment to the next.
- A *transformation primitive* (TP) p_i describes *what happened* over a short stretch of time — the change, not the action.

- *Reusable domain variables* Z_D and *directed relation operators* R_D explain the recurring parts of those changes.
- Together these form a *relational latent world* $W_D = (Z_D, R_D)$: a compact model of "how this domain behaves."

Real-world example. A manipulation robot should not memorise one unrelated motion for every scene. It should learn reusable structure such as "side contact moves a free object" and "a wall constrains that motion," then apply the same mechanisms and relations to a new object.

This world closes a learning loop: when it is confident, it asks for missing evidence; when it is not, it asks to try transformations at the edge of what it currently understands. In parallel, several demonstrations of the same task are used to learn a *task constraint* C_T: a compact description of the goal, learned by predicting one demonstration from another. The task state is deliberately minimal, $Z_T = (S_t, C_T)$. The task is not stored as a new object; instead C_T *acts on* the explicit world W_D to produce a distribution over "next transformations that fit the task," plus a "should I stop now" signal. Low-level control is then obtained by asking the already-learned model the reverse question: *"which action would produce that transformation?"*

There is no separate desired-transformation variable D_T, no explicit task graph G_T in the first implementation, no learned uncertainty object U_D, and no separate exploration or task motor policy. The paper keeps these out deliberately, and says why.

The paper also places this proposal against related ideas — object-centric learning, equivariant representations, temporal abstraction, causal representation learning, relational inductive bias, active experiment design, task inference, imitation learning, and model-based control. For each, it states plainly what is borrowed, what is changed, and why existing guarantees do not carry over. The final product is a *falsifiable* research programme: world variables must earn their status through reusable predictions; task context must earn its status through transfer across demonstrations; and every abstraction is judged by its held-out consequences, not by whether its learned "shape" looks clean.

1. Scientific Problem Definition

1.1 The scientific question

Let an agent observe O_t, carry out a low-level action a_t, and receive the next observation O_{t+1}. The scientific question is:

> Can the agent use time-aligned observation–action streams to discover reusable effective variables and their directed relations, actively gather the evidence needed to refine that structure, infer a persistent task constraint from different demonstrations of the same task, and turn that constraint back into actions — *without ever being told a human vocabulary* of objects, mechanisms, or named actions?

Real-world example. The robot is not told which latent unit means "movable object" or which mechanism means "sliding contact." It sees the scene before acting, the executed motor command, and the scene afterwards, and must discover which recurring state components and effects matter.

The primitive unit of data is a continuous interaction trajectory

$$\tau = (O_0, a_0, O_1, a_1, \ldots, a_{T-1}, O_T),$$

with strict alignment in time between what was observed and what was actually done.

The proposal rejects two shortcuts. First, an *action label* is not the same as the *transformation* it produces: the same action can have different effects in different situations, and different actions can produce the same effect. Second, a *task label* or a *final picture* is not a sufficient description of a task: different strategies can satisfy the same goal, while identical endpoints can hide different required processes.

Real-world example. "Push the cup" is one action, but the *change* depends on the cup: an empty cup slides, a full cup tips, a glued cup does not move. Conversely, "slide the cup left" can be achieved by pushing from the right or pulling from the left. So what the agent should record is the *change*, not the name of the action or one fixed motor sequence.

1.2 Target explanatory hierarchy

The proposed hierarchy is

$$\boxed{O_t \to S_t \to TP \to \begin{cases} Z_D \leftrightarrow R_D \to W_D \to \text{exploration}, \\ TP \text{ sequences} \to C_T \to \text{task transformation} \end{cases} \to \text{intervention}.}$$

Reading left to right: raw observation → a persistent state → a "what changed" summary (TP). That summary then feeds two branches: (1) reusable mechanisms and their relations form a world model, used for exploration; (2) sequences of changes are used to learn a task goal. Both branches end back in a concrete action. The whole loop is "see, act, learn, act better."

The terms play deliberately different roles:

- S_t is a persistent coordinate system for interaction-relevant state.
- p_i is an experience-level description of one concrete local change over a variable-length stretch of time.
- $Z_D = \{Z_D^j\}$ is a domain-level bank of reusable effective mechanisms; z_i^j is the value of mechanism j in experience i.
- $R_D = \{R_D^{j \to k}\}$ is a set of stable directed operators describing how one mechanism changes another.
- $W_D = (Z_D, R_D)$ is the shared relational latent world for the chosen domain and scale.
- C_T is a persistent task constraint inferred from a TP sequence and trained to transfer across different demonstrations of the same task.
- $Z_T = (S_t, C_T)$ is the minimal task state. W_D stays an external, domain-shared structure rather than being folded into Z_T.

Task transformation is *not* a new latent variable. It is the operation

$$\boxed{\mathcal{T}_T : (S_t, W_D, C_T) \longmapsto P_T(p_{\text{next}}, y^{\text{stop}} \mid S_t, W_D, C_T),}$$

possibly also looking at the most recent TP as a short-term "progress cue." In plain terms, C_T does not create a new world; it *re-weights* the transformations that W_D already makes possible, keeping the ones that fit the goal. The first implementation therefore has neither a stored "desired next transformation" D_T nor a separately built task graph G_T.

1.3 Why this problem is not ordinary representation learning

Predicting the next observation is not enough to pin down the right concepts: many different internal coordinate systems can reproduce the same one-step predictions. So this paper asks a stronger, functional question. Which variables keep explaining effects *across* contexts? Which directed relations stay necessary under controlled comparisons? Which task code changes future transformation predictions when the world state is held the same? A latent variable is accepted not because it looks interpretable or statistically separated, but because it *earns operational meaning* by being reusable.

Real-world example. A robot could memorise "this exact block, pose, push, and displacement," yet fail in every new scene. A useful mechanism instead predicts how pushing transfers across new object poses, force magnitudes, and surrounding constraints.

This also changes the role of action. Action is not merely a control target. It is *experimental leverage*. Different actions supply the contrasts that passive watching cannot, and the learned structure in turn tells the agent which action would be informative next. The world model is therefore both an explanation and an experiment-design state.

1.4 Scope and non-claims

The framework targets a bounded domain, sensor suite, action space, spatial scale, temporal scale, and abstraction level. It does not claim to recover fundamental physical variables, a unique object ontology, or a universally valid causal graph. Z_D is an *effective* description relative to what the agent can observe and change. Relations are functional modulation operators in latent effect space, not automatically causal effects in the formal interventionist sense.

Four kinds of statements are kept strictly separate throughout:

1. **Physical or structural prior:** a claim about regularity in the domain or task family.
2. **Identifiability condition:** a condition on the data needed to tell hypotheses apart.
3. **First implementation:** an architectural, distributional, or optimisation choice.
4. **Diagnostic:** an evaluation or evidence quantity that is not promoted into a latent object or a strong training loss.

This distinction matters. A Transformer, a Gaussian posterior, a Hard-Concrete relaxation, pairwise message passing, task grouping supplied by a dataset, or local differentiability of a learned controller is *not* a physical assumption.

Real-world example. "We used a Transformer" describes an implementation, not the physics of manipulation. A different architecture could express the same structural claim, so predictive success alone cannot turn the architecture into a physical assumption.

2. Literature Review and Research Positioning

This section says, for each neighbouring field: "here is the useful idea we take, here is the important way we differ, and here is a guarantee from that field that does not *carry over to us." You can read just the first sentence of each subsection to get the idea; the detail is there if you want it.*

2.1 Object-centric and temporally persistent representations

Slot Attention introduced exchangeable "slots" that bind perceptual features into a set of task-dependent representations [1]. SAVi added temporal conditioning for video, showing that earlier slots can guide binding in later frames [2]. SOLD showed the value of slot-structured latent dynamics for relational manipulation and model-based reinforcement learning [3]. We borrow the set-valued latent interface, temporal conditioning, and action-conditioned dynamics. We differ in ontology and purpose: a slot here is a persistent *interaction unit*, not a claim that the world is made of objects, and slots are only the coordinate system from which transformations and mechanisms are discovered. Consequently, object-discovery results, segmentation quality, or empirical control gains do not guarantee TP, Z_D, or R_D identifiability. Slot permutation, splitting, merging, and non-object solutions remain possible.

Vision Transformers provide a convenient localised token interface [4], and Set Transformers and Deep Sets provide permutation-aware computation over sets [18,19]. These architectures are borrowed as implementation tools. No theorem about the world follows from choosing them, and no representation guarantee transfers from their universal-approximation or attention properties.

2.2 Invariance, equivariance, and interaction-grounded change

Equivariant representation learning formalises "predictable latent change under known transformation groups." SIE explicitly splits invariant and equivariant components and stops equivariant information from collapsing into the invariant branch [29]. Symmetry-based approaches clarify an important goal: irrelevant variation should be thrown away while action-relevant change stays predictable. We borrow this separation of stable content from structured change. We differ because robot interventions need not form a known group, be invertible, or act globally; contact, containment, irreversible change, and state-dependent effects all violate the clean symmetry setting. The proposed TP is therefore

learned from *realised* state change and executed actions, not from supervised relative group elements. Group-equivariance, losslessness, and disentanglement guarantees do not transfer.

This difference is substantive. The target is not "invariance to action" but a representation in which

$$\boxed{(S, A) \mapsto p \mapsto \Delta S}$$

is stable enough to support mechanism discovery and inversion. What stays invariant is the reusable *mechanism identity*; what varies (equivariantly, in a loose operational sense) is its *realisation and effect*.

Real-world example. The mechanism "lateral contact produces sliding" can recur across objects, while the displacement varies with force, friction, and object mass. The representation should keep the reusable law fixed while allowing its realised effect to vary.

2.3 Temporal abstraction, skills, and transformation primitives

The options framework formalised temporally extended actions in semi-Markov decision processes [5]. Temporal variational inference and later temporally abstract world models learn latent skills or options from trajectories [6,30]. We borrow variable duration, learned termination, and the principle that temporal segmentation should reflect a coherent latent regime. We differ in what the segment's latent *denotes*. A skill or option is normally a policy-side object — *how to act* — whereas a TP denotes the realised world-side transformation — *what happened*. The TP posterior therefore excludes action, while its prospective prior includes state and intervention. Option-value, policy-composition, and semi-MDP guarantees do not establish that the inferred TP is action-invariant, mechanism-reusable, or causally meaningful.

Real-world example. A skill may be a particular grasp-and-lift motion, whereas a transformation is "the object is now above the table." Different motions can produce that change, and the same motion can fail on a heavier object; the model therefore represents the change itself.

Latent dynamics and predictive world models show that useful representations can be learned through future prediction and used for planning [3,26–28,31]. Predictive invariant/equivariant world models further motivate separating stable content from predictable change [7]. Latent-action work shows both the promise of discovering action-like coordinates and the risk that distractors or missing intervention information contaminate them [8,9]. We borrow action-conditioned latent prediction and receding-horizon optimisation, but keep the executed intervention explicit and distinguish the *effect* posterior from the *intervention-conditioned* prior. We also insert explicit transformation, mechanism, and relation levels, and attribute errors across them. Predictive accuracy or latent-action cycle consistency alone does not identify the proposed hierarchy; a monolithic model can fit transitions while hiding exactly the structure we care about.

2.4 Disentanglement and causal representation learning

Unsupervised disentanglement is non-identifiable without inductive biases on the model and data [32]. Nonlinear ICA becomes identifiable only with additional structure such as observed auxiliary variables [33]. These negative and positive results motivate the explicit statement of priors and coverage conditions in this paper. We do not claim that sparsity or prediction uniquely recovers true factors.

CITRIS uses temporal sequences with observed intervention targets to identify multidimensional causal factors under its generative assumptions [10]. BISCUIT studies unknown binary interaction indicators that switch causal mechanisms [11]. Mechanism-sparsity work shows that sparse latent dependencies or intervention masks can yield identifiability under graph and data conditions [12]. Multi-node and linear-system intervention analyses relax or clarify intervention requirements in more restricted mixing settings [13,14], while interventional causal representation learning derives identifiability from support changes under interventions [15]. iCITRIS extends temporal causal representation learning to instantaneous effects [16]. We borrow four principles: temporal information, intervention diversity, sparse or structured mechanism participation, and the need to state what variation identifies an edge.

The differences are equally important. The present model uses learned slots, variable-duration TPs, unknown many-to-many mechanism allocation, nonlinear mechanism modules, and directed residual relation operators. Intervention targets are not supplied. Z_D is effective and domain-relative, not assumed to equal a ground-truth causal variable. The learned relation graph need not be acyclic. Hence the identifiability theorems of [10–16,33] do not transfer. They motivate diagnostics and experimental designs, not a proof that this neural hierarchy recovers a unique causal model.

Real-world example. Many causal-representation results assume a label such as "the drawer joint was intervened on." Here the robot sees only its motor command and the resulting scene change, so guarantees that require intervention targets do not directly transfer.

2.5 Reusable mechanisms, latent feature allocation, and relational structure

The Indian Buffet Process supplies a conceptual precedent for sparse, many-to-many latent feature allocation [17]. We borrow the idea that one experience may activate several reusable features and one feature may recur across many experiences. We do not use the IBP generative process or its nonparametric inference guarantees; the first implementation uses an overcomplete finite bank and differentiable participation gates.

Graph networks and relational inductive biases motivate representing entities and their interactions explicitly [22]. FiLM supplies a simple conditioning operator [21], and L_0 regularisation supplies differentiable sparse gates [20]. We borrow directed message passing, constrained modulation, and sparse structural selection. We differ in edge semantics: nodes are reusable effective mechanisms and an edge is the residual modulation of a target mechanism by a source mechanism. The graph is not an input object graph, and graph-network expressivity does not make an edge identifiable. Direction requires matched-context contrasts and held-out explanatory power.

2.6 Active learning, experimental design, and curiosity

Bayesian experimental design defines experiment value through expected information gain [23], while D-optimal and log-determinant criteria reward measurements that expand rather than duplicate supported directions [24]. Curiosity methods reward prediction error as an intrinsic signal [25]. We borrow the principle that data collection should be driven by expected reduction of structural ambiguity, and use an explicit log-determinant coverage statistic as a first estimator.

We differ from generic curiosity in two ways. First, novelty is not intrinsically valuable: evidence must bear on mechanism reuse or directional relation identification. Second, disagreement is rewarded only when the lower TP level is locally competent; otherwise the agent would chase sensor noise or exploit its own predictor. We also do not maintain a full Bayesian posterior over a neural world model. Therefore Bayesian optimality, submodularity, and regret guarantees do not transfer to the learned nonlinear objective.

Real-world example. A robot rewarded for any prediction error may repeatedly inspect sensor flicker. Structural exploration instead favours an action such as probing a blocked drawer when that outcome can distinguish competing mechanism hypotheses.

2.7 Task representations from demonstrations

One-shot visual imitation and Task-Embedded Control Networks infer a task representation from demonstrations and condition a controller on it [34,35]. Probabilistic-context meta-RL separates task inference from control [36]. Conditional Neural Processes separate a context set from target prediction [37]. We borrow the context/query logic: infer C_T from one demonstration and require it to predict *another* demonstration of the same task. This is stronger than reconstructing its own trajectory, because it discourages a path-specific code.

The proposed task variable differs from prior task embeddings in both supervision and interface. The first experiment receives episode boundaries and same-task grouping, but *not* semantic task labels as

inputs to the network. C_T predicts transformations rather than direct motor commands or rewards. It acts on the explicit W_D, and the low-level action is found by TP-prior inversion. Meta-learning generalisation, imitation success, or neural-process consistency does not imply that C_T has captured a *task constraint* instead of strategy, initial state, demonstrator identity, or dataset bias. That claim must be tested with cross-strategy demonstrations, cross-initial-state transfer, and a no-C_T predictive ablation.

Real-world example. Two demonstrations may place a cup in a tray using different grasps and approach paths. A useful task code should capture the shared constraint, not the demonstrator's particular trajectory, and should predict progress along the other strategy.

2.8 Multimodal task progression and model-based action realisation

Demonstrations of the same task can contain several valid next transformations. A unimodal regressor can average incompatible modes and pressure the task encoder to leak strategy identity. Mixture-density models provide a direct probabilistic representation of multimodal conditional outcomes [38]; diffusion policies show the practical importance of multimodal action distributions in robot imitation [39]. We borrow only the need for a multimodal output and choose a conditional Gaussian mixture in TP space for the first implementation. Diffusion-policy control performance and density-model expressivity do not guarantee that the modes correspond to meaningful task alternatives.

Real-world example. To place an object behind an obstacle, the robot may pass it on the left or on the right. Averaging the two next transformations can point into the obstacle, whereas a mixture keeps both valid strategies separate.

Differentiable MPC, PETS, visual foresight, and Universal Planning Networks optimise actions through learned predictive models [26–28,40]. We borrow direct action-sequence optimisation, uncertainty-aware or sampling baselines, and receding-horizon replanning. The difference is the objective: exploration maximises structural evidence and abstraction challenge; task execution matches the TP distribution demanded by $\mathcal{T}_T$. No independent motor policy is learned. Classical MPC guarantees do not transfer because the learned TP and world abstractions are approximate, stochastic, and only locally calibrated.

2.9 Positioning summary

Line of work	Borrowed or inspired	Distinctive move here	Guarantee that does not transfer
Object-centric video	persistent set-valued state	slot as interaction unit, not object ontology	object recovery or slot identifiability
Equivariance	separate stable identity from structured change	interventions need not be known groups	group-equivariance and losslessness
Temporal abstraction	variable duration and termination	TP represents realised effect, not a policy option	semi-MDP or option-control guarantees
Causal representation learning	interventions, sparsity, contrast, identifiability discipline	unknown targets, learned TP mediator, effective mechanisms, possibly cyclic relations	recovery of true causal variables/graph
Graph networks	explicit directed interactions	edge is source-to-target mechanism modulation	edge semantics or direction identifiability
Experimental design	non-redundant evidence gain	structure-specific evidence plus TP-gated abstraction frontier	Bayesian optimality or submodularity
Task/meta learning	demonstration-derived context	cross-demonstration TP prediction on explicit W_D	strategy-invariant task identity
Model-based control	optimise actions through predictions	invert TP prior for both exploration and tasks	stability or optimality under model error

3. Physical and Structural Priors

A "prior" here means an assumption we make before *looking at data — a belief about how the world is organised. We list only assumptions about the domain itself here. Choices about networks, distributions, and optimisers come later. Each prior gets a plain-language restatement and an example.*

P1. Domain-local effective compressibility

At a specified domain, spatial scale, temporal scale, and abstraction level, intervention-relevant change can be explained by fewer reusable effective mechanisms than raw observation dimensions:

$$\boxed{\Delta S \approx F_W(S; Z_D, R_D), \qquad |Z_D| \ll \dim(O).}$$

A small number of reusable causes can explain most of what changes, even though the raw measurements (pixels, readings) are huge in number.

Real-world example. A robot camera stream contains millions of pixel values, yet a useful account may say "the block moved because of contact and was constrained by the rail." P1 assumes that such compact summaries of interaction exist.

The mechanisms are not asserted to be fundamental physical variables or unique across domains.

P2. Intervention–response regularity

The effect of a low-level intervention depends systematically on the pre-intervention state:

$$\boxed{P(S_{t+1} \mid S_t, a_t) \neq P(S_{t+1} \mid a_t) \text{ in general.}}$$

What an action does depends on where you are. The same action in different states generally gives different outcomes.

Real-world example. The same gripper displacement may slide a free block, rotate a hinged object, or do nothing to a fixed object. The action is the same, but its effect depends on state.

Comparable transformations can recur across different controls, and comparable controls can yield different transformations under different states. This regularity is what makes a transformation variable distinct from an action code.

P3. Temporally local transformation regimes

Over some bounded intervals, a single latent transformation description is sufficient to explain the local evolution better than re-describing the code at every timestep. Boundaries need not coincide with action switches and need not be perfectly discrete in every domain. The prior is local persistence, not a preferred duration.

Real-world example. "Sliding the block along the rail" is one coherent transformation even when executed through many small control steps. Treating every step as a new phenomenon would lose the shared change pattern.

P4. Reusable and locally compositional mechanisms

The same mechanism function can recur across contexts and tasks, while a local TP normally engages a subset of the mechanisms available in the domain:

$$\boxed{e_i^j = \mathcal{M}_j(S_i^{\text{pre}}, v_i^j), \qquad \mathbb{E}\|m_i\|_0 < M_{\max}.}$$

Each mechanism is a reusable function. In any single change, only a few *mechanisms are usually active, and one change can activate several at once (many-to-many).*

Real-world example. The mechanism "contact force produces translation" recurs across many objects. One TP may combine it with rail constraint, while the same contact mechanism appears in many other TPs; this many-to-many reuse is what P4 assumes.

This does not impose one TP per mechanism. The mapping is many-to-many.

P5. Stable directed relational modulation

Some effects cannot be explained by isolated mechanisms alone but admit reusable, directed source-to-target modulation:

$$\boxed{R_D^{j\to k} \neq R_D^{k\to j} \text{ in general.}}$$

Some effects need two mechanisms plus a direction *between them. "A changes B" is not the same as "B changes A."*

Real-world example. A closed gripper may transmit the arm's motion to an object, while the object's motion does not symmetrically command the arm. P5 allows this directed modulation; whether the data identify the direction remains an empirical question.

Relation existence is a property of the domain-level mechanism system; whether the current dataset supports it is an epistemic question.

P6. A shared domain world under task variation

Within the chosen domain, tasks normally change *which* feasible transformations are relevant, not the underlying domain mechanisms themselves. Thus W_D is shared, while C_T reweights its currently realisable transformation possibilities. This prior motivates freezing the world representation when first learning task context.

Real-world example. The manipulation domain does not change between "place the cup in the tray" and "move the cup away from the tray." The same world mechanisms apply; the task constraint selects different transformations from them.

P7. Cross-realisation persistence of task constraint

Different successful trajectories can instantiate the same task constraint. Therefore a task has information that persists across variation in initial state, strategy, and low-level execution:

$$\boxed{\tau_{g,r} \sim g,\ \tau_{g,s} \sim g \quad \Rightarrow \quad C_T^{g,r} \text{ and } C_T^{g,s} \text{ are functionally interchangeable.}}$$

Two different successful attempts at the same task should give task codes that do the same job, even if their numbers differ.

Real-world example. Two robots may open the same drawer using different grasps and trajectories. Their task codes are functionally equivalent if each predicts the other's task progression, even when the code coordinates are not numerically identical.

P8. Task relevance is expressible in transformation space

For the first target domains, the information needed to tell task-consistent from task-inconsistent progress can be expressed as a distribution over TPs and termination conditional on the current world state. This is a genuine scope prior. If two tasks demand different information that never changes their TP progression or stop condition, the proposed C_T interface cannot distinguish them.

What is deliberately not a physical or structural prior

The following are implementation or experimental choices: ViT features, slots, Transformers, FiLM, diagonal Gaussians, Hard-Concrete gates, additive first-order effect composition, pairwise relations, kernel bandwidths, grouped task demonstrations supplied by the dataset, gradient optimisation, and thresholded TP competence. Their role is to make the first test finite and interpretable. Failure of one such choice does not by itself falsify P1–P8; failure across credible implementations and appropriate coverage does.

Real-world example. If one network fails to learn reusable contact mechanics, that may expose an implementation limit rather than prove that such mechanisms do not exist. A structural prior is rejected only after adequate data and reasonable model alternatives also fail.

4. Mathematical Modeling

Each displayed equation is followed by one short plain-language explanation.

4.1 Data, notation, and the two learning branches

World-discovery data are temporally aligned interaction streams:

$$\mathcal{D}_{\text{world}} = \{(O_0, a_0, \ldots, a_{T-1}, O_T)\}\,.$$

In plain terms. This defines the observation–action streams used to learn the world.

No object category, semantic action, TP boundary, Z_D, R_D, mechanism, or causal-graph label is required for training. Simulation ground truth may be retained for evaluation only.

Task data provide episode boundaries and same-task grouping:

$$\boxed{\mathcal{D}_{\text{task}} = \{\mathcal{T}_g\}_{g=1}^{G}, \qquad \mathcal{T}_g = \{\tau_{g,1}, \ldots, \tau_{g,R_g}\}, \quad R_g \geq 2.}$$

In plain terms. This groups at least two demonstrations that share the same task.

This grouping is dataset supervision for the first theory test, not a claim that task equivalence is already solved. The branches share S, TP, and W_D:

$$\boxed{\mathcal{D}_{\text{world}} \to S \to TP \to W_D, \qquad \mathcal{D}_{\text{task}} \to TP \text{ sequences} \to C_T.}$$

In plain terms. This separates world learning from task learning while making both reuse the same transformations.

4.2 Interaction representation

Temporally persistent latent state

Visual tokens are

$$H_t = E_{\text{vis}}(O_t) = \{h_t^1, \ldots, h_t^N\}.$$

In plain terms. This converts the raw observation into visual tokens.

The latent state is a bounded-capacity set

$$\boxed{S_t = E_S(H_t, S_{t-1}) = \{s_t^1, \ldots, s_t^{K_{\max}}\},}$$

In plain terms. This updates a small set of persistent state slots from the current and previous observations.

with optional activity variables. The executed action is *not* input to E_S; state inference and intervention remain distinct. The first implementation uses a ViT-style token encoder and a SAVi-like recurrent Slot Attention module. "Slot" denotes a persistent latent interaction unit.

Real-world example. One slot may keep tracking a cup while another tracks a drawer handle. Persistent slot identity lets the model measure how each interaction unit changed across time.

Action-conditioned dynamics and temporal correspondence

The transition model is

$$\boxed{\widehat{S}_{t+1} = F_S(S_t, a_t).}$$

In plain terms. This predicts the next latent state from the current state and executed action.

The low-level control should be the command actually executed, for example

$$a_t = (\Delta x_t, \Delta y_t, \Delta z_t, \Delta r_t, g_t),$$

In plain terms. This represents the action using the low-level command actually sent to the robot.

rather than a semantic label such as "push". A small MLP embeds a_t; a permutation-equivariant slot Transformer processes S_t; and FiLM injects the action condition:

$$h_t^a = E_a(a_t), \qquad (\gamma_t, \beta_t) = G_a(h_t^a), \qquad \tilde{s}_t^k = \gamma_t \odot \bar{s}_t^k + \beta_t.$$

In plain terms. This embeds the action and uses it to modulate every state slot.

Residual slot-order ambiguity is resolved during training by

$$\pi_t^* = \arg\min_{\pi} \sum_{k=1}^{K_{\max}} D_s(\widehat{s}_{t+1}^k, s_{t+1}^{\pi(k)}),$$

In plain terms. This finds the best correspondence between predicted and observed slots.

yielding

$$\boxed{L_{\text{dyn}} = \sum_{t,k} D_s(\widehat{s}_{t+1}^k, s_{t+1}^{\pi_t^*(k)}).}$$

In plain terms. This trains state dynamics using the matched slot pairs.

The matched change is

$$\Delta s_t^k = s_{t+1}^{\pi_t^*(k)} - s_t^k, \qquad \Delta S_t = \{\Delta s_t^k\}_{k=1}^{K_{\max}}.$$

In plain terms. This measures how each matched slot changed during one step.

The minimal interaction-representation objective is

$$\boxed{L_{\text{IR}} = L_{\text{dyn}}.}$$

In plain terms. This makes next-state prediction the core objective of interaction representation.

A weak observation reconstruction term $\lambda_{\text{rec}} L_{\text{rec}}$ may be used only to diagnose or prevent collapse. Strong reconstruction is avoided because it can preserve appearance detail irrelevant to intervention-induced change.

4.3 Transformation primitives

Experience and variable duration

For segment i with boundaries $b_i < e_i$,

$$\boxed{\mathcal{I}_i = (S_i^{\text{pre}}, A_i, S_i^{\text{post}}), \quad S_i^{\text{pre}} = S_{b_i}, \quad S_i^{\text{post}} = S_{e_i}, \quad A_i = a_{b_i:e_i-1}.}$$

In plain terms. This packages one variable-length segment with its start, end, and action sequence.

After matching, $\Delta S_i = S_i^{\text{post}} - S_i^{\text{pre}}$. A candidate duration satisfies $d_t \in \{1, \ldots, H_{\max}\}$. Transition tokens

$$x_\tau = E_b(S_\tau, a_\tau, \Delta S_\tau)$$

In plain terms. This encodes each transition into a token for boundary detection.

are processed by a temporal Transformer. A hazard head produces $b_{t+j} = \sigma(H_b(h_{t+j}))$ and

$$\boxed{P(d_t = j) = b_{t+j} \prod_{r<j} (1 - b_{t+r}).}$$

In plain terms. This gives the probability that the current transformation ends after exactly j steps.

The boundary is meaningful only if one TP remains sufficient inside the segment and ceases to be sufficient beyond it.

Real-world example. "The block slides freely" can be one transformation. When it contacts a wall and begins rotating, the previous description no longer fits, providing evidence for a new TP boundary.

Posterior: what happened

The transformation posterior excludes action:

$$\boxed{q_\phi(p_i \mid S_i^{\text{pre}}, S_i^{\text{post}}).}$$

In plain terms. This infers what transformation occurred from the states before and after it.

Matched slot pairs form tokens

$$x_i^k = E_\Delta(s_{i,\text{pre}}^k, \Delta s_i^k).$$

In plain terms. This describes the change of each state slot in its starting context.

A Set Transformer and TP query pool the multi-slot change:

$$H_i^\Delta = \text{SetTransformer}_{\text{post}}(\{x_i^k\}), \qquad h_i^{\text{post}} = \text{CrossAttn}(q_{\text{TP}}, H_i^\Delta).$$

In plain terms. This combines all slot changes into one transformation summary.

The first posterior is diagonal Gaussian,

$$q_\phi = \mathcal{N}(\mu_i^q, \mathrm{diag}((\sigma_i^q)^2)), \qquad p_i = \mu_i^q + \sigma_i^q \odot \epsilon.$$

In plain terms. This samples a trainable TP representation from the posterior distribution.

Gaussianity is an implementation baseline, not a world prior.

Prior: what an intervention is expected to cause

The prospective TP prior is

$$\boxed{p_\theta(p_i \mid S_i^{\mathrm{pre}}, A_i).}$$

In plain terms. This predicts the transformation expected from a state and action sequence.

A Transformer encodes the variable-length action sequence, a set encoder summarises the pre-state, and a fusion network predicts $(\mu_i^p, \log \sigma_i^p)$:

$$h_i^A = E_A(A_i), \quad h_i^S = E_{\mathrm{set}}(S_i^{\mathrm{pre}}), \quad h_i^{\mathrm{prior}} = F_{\mathrm{prior}}(h_i^S, h_i^A).$$

In plain terms. This combines the starting state and action sequence to parameterise the TP prior.

The prior is also diagonal Gaussian in the first implementation. Posterior and prior are aligned by

$$\boxed{L_{\mathrm{align}} = D_{\mathrm{KL}}\Big[q_\phi(p_i \mid S_i^{\mathrm{pre}}, S_i^{\mathrm{post}}) \| p_\theta(p_i \mid S_i^{\mathrm{pre}}, A_i)\Big].}$$

In plain terms. This makes the predicted TP agree with the TP inferred from the real outcome.

This objective permits the two contrasts essential to the intended semantics:

$$S^{(1)} \neq S^{(2)},\ A^{(1)} \approx A^{(2)} \Rightarrow p^{(1)} \not\approx p^{(2)} \text{ when effects differ,}$$
$$A^{(1)} \neq A^{(2)},\ \Delta S^{(1)} \approx \Delta S^{(2)} \Rightarrow p^{(1)} \approx p^{(2)}.$$

In plain terms. These contrasts make TPs follow effects rather than action labels.

Real-world example. Contrast (1): the same gripper motion may slide a free block but fail on a fixed block, so the TPs differ. Contrast (2): pushing from the right and pulling from the left may produce the same leftward slide, so their TPs should agree.

TP effect and boundary semantics

The TP must explain latent state change:

$$\widehat{\Delta S}_i^{TP} = F_{\mathrm{TP}}(S_i^{\mathrm{pre}}, p_i), \qquad \boxed{L_{\mathrm{effect}}^{TP} = D_S(\widehat{\Delta S}_i^{TP}, \Delta S_i).}$$

In plain terms. This decodes the TP into a state change and compares it with what actually happened.

The first decoder is a TP-conditioned set-equivariant slot Transformer. For each time inside a segment,

$$\widehat{\Delta S}_{i,t}^{TP} = F_{\mathrm{TP}}(S_t, p_i),$$

In plain terms. This uses the same TP to predict every change inside its segment.

and

$$\boxed{L_{\text{seg}} = \sum_i \sum_{t=b_i}^{e_i-1} D_S(\widehat{\Delta S}_{i,t}^{TP}, \Delta S_t).}$$

In plain terms. This checks that one TP remains valid throughout the whole segment.

A weak minimum-description-length pressure prevents a boundary at every timestep:

$$L_{\text{complexity}} = N_{\text{segments}} \quad \text{or} \quad L_{\text{complexity}} = \sum_t b_t.$$

In plain terms. This discourages the model from creating unnecessary boundaries.

The TP objective is

$$\boxed{L_{\text{TP}} = L_{\text{effect}}^{TP} + \lambda_A L_{\text{align}} + \lambda_S L_{\text{seg}} + \lambda_C L_{\text{complexity}}.}$$

In plain terms. This combines TP accuracy, prior alignment, segment consistency, and boundary simplicity.

During this stage L_{TP} may update E_S, allowing interaction units and transformation coordinates to co-adapt. TP dimensionality is selected by held-out prediction, reuse, and cross-context transfer; compactness is not assumed for aesthetic reasons.

4.4 From TPs to the relational latent world W_D

TP experience and many-to-many mechanism allocation

Every TP retains its provenance:

$$\boxed{\mathcal{E}_i = (S_i^{\text{pre}}, A_i, S_i^{\text{post}}, \Delta S_i, p_i).}$$

In plain terms. This stores one complete interaction and the transformation it produced.

Let $M_{\max}$ be an overcomplete capacity, not the assumed true number of mechanisms:

$$Z_D = \{Z_D^1, \ldots, Z_D^{M_{\max}}\}.$$

In plain terms. This creates more candidate mechanisms than may be needed, so unused ones can be removed by learning.

For experience i, $m_i^j \in \{0,1\}$ indicates participation, $v_i^j \in \mathbb{R}^{d_v}$ is the realisation conditional on participation, and

$$\boxed{z_i^j = m_i^j v_i^j.}$$

In plain terms. This combines whether a mechanism is active with the value it takes when active.

The distinction is necessary because $v_i^j = 0$ cannot otherwise distinguish inactivity from an active zero-valued realisation. The abstraction posterior is

$$\boxed{q_\eta(m_i, v_i \mid p_i, S_i^{\text{pre}}).}$$

In plain terms. This infers which mechanisms produced the observed transformation and their values.

Real-world example. For a contact mechanism, m records whether contact participated and v records its realised strength or direction. No contact is therefore distinct from active contact whose measured displacement is momentarily zero.

Learned mechanism queries q_D^j attend to an embedded TP and pre-state slots:

$$X_i = \{E_p(p_i), E_s(s_{i,\text{pre}}^1), \ldots, E_s(s_{i,\text{pre}}^{K_{\max}})\},$$

In plain terms. This gathers the transformation and its starting context into one input set.

$$\boxed{h_i^j = \text{CrossAttn}(q_D^j, X_i).}$$

In plain terms. Each mechanism query extracts the evidence relevant to that mechanism.

Independent heads predict participation and realisation. The first implementation uses

$$m_i^j \sim \text{HardConcrete}(H_m(h_i^j)),$$

In plain terms. This makes a trainable, nearly binary decision about whether mechanism j participates.

and

$$v_i^j = \mu_{v,i}^j + \sigma_{v,i}^j \odot \epsilon, \qquad (\mu_{v,i}^j, \log \sigma_{v,i}^j) = H_v(h_i^j).$$

In plain terms. This samples the mechanism's value while keeping the sampling step trainable.

There is no categorical competition because several mechanisms may participate in one TP.

Reusable mechanism functions

Each candidate variable owns a reusable mechanism identity $\mathcal{M}_j$. Its effect in experience i is

$$\boxed{e_i^j = m_i^j \mathcal{M}_j(S_i^{\text{pre}}, v_i^j),}$$

In plain terms. This predicts mechanism j's contribution to the state change.

where e_i^j lies in slot-change space. The first network is a shared set-dynamics Transformer with mechanism-specific query conditioning and small adapters. A shared backbone supplies generic state processing; the query, realisation, and adapter preserve mechanism identity.

Before relation learning, effects compose additively:

$$\boxed{\widehat{\Delta S}_i^Z = \sum_{j=1}^{M_{\max}} e_i^j,} \qquad \boxed{L_{\text{effect}}^D = \sum_i D_S(\Delta S_i, \widehat{\Delta S}_i^Z).}$$

In plain terms. This adds the mechanism effects and compares the sum with the observed change.

Real-world example. Predicted change may begin as the sum of pushing and constraint effects. Any systematic residual, such as the constraint redirecting the push into rotation, is left for the relation layer to explain.

An optional low-capacity decoder reconstructs the TP from active mechanism realisations:

$$y_i^j = m_i^j E_Z(q_D^j, v_i^j), \quad h_i^P = \text{CrossAttn}(q_P, \{y_i^j\}), \quad \widehat{p}_i = G_P(h_i^P),$$

In plain terms. This reconstructs the TP from the active mechanisms as a consistency check.

$$\boxed{L_{\mathrm{TP-rec}} = D_P(p_i, \widehat{p}_i).}$$

In plain terms. This measures how much information was lost when reconstructing the TP.

This is a weak hierarchical consistency term, not the primary grounding objective. Sparse local participation is encouraged by

$$\boxed{L_{\mathrm{participation}} = \sum_{i,j} \mathbb{E}[\mathbf{1}(m_i^j \neq 0)].}$$

In plain terms. This penalises using too many mechanisms at once.

The Phase-I objective is

$$\boxed{L_{Z_D}^{(0)} = L_{\mathrm{effect}}^{D} + \lambda_P L_{\mathrm{participation}} + \lambda_T L_{\mathrm{TP-rec}},}$$

In plain terms. This is the first-stage loss: predict change, stay sparse, and weakly preserve the TP.

with λ_T weak or zero. A useful mechanism is not one that is merely frequent; it must use the same $\mathcal{M}_j$ to predict held-out effects across diverse contexts and TP combinations.

Preserving the TP mediation

Prospective mechanism engagement must pass through the TP prior:

$$\boxed{\widetilde{q}_\eta(m, v \mid S, A) = \int q_\eta(m, v \mid p, S) p_\theta(p \mid S, A)\, dp.}$$

In plain terms. This predicts mechanism engagement through possible TPs, preventing an action-to-mechanism shortcut.

Monte Carlo estimation draws $p^{(l)} \sim p_\theta(p \mid S, A)$ and then $(m^{(l)}, v^{(l)}) \sim q_\eta(\cdot \mid p^{(l)}, S)$. Expected engagement is

$$\boxed{\rho_j(S, A) \approx \frac{1}{L} \sum_{l=1}^{L} m_j^{(l)}.}$$

In plain terms. This estimates how likely the action is to engage mechanism j.

A direct shortcut $(S, A) \to Z_D$ is excluded, because it would allow Z_D to become a second action embedding and would sever the proposed explanatory hierarchy.

Directed relations and the corrected gate placement

The domain relation set is

$$\boxed{R_D = \{R_D^{j \to k}\}_{j \neq k}.}$$

In plain terms. This lists all candidate directed relations between different mechanisms.

$R_D^{j \to k}$ is a reusable operator by which source mechanism j modifies the effect expressed by target mechanism k. It is neither co-occurrence nor a separate effect generator. Let

$$\mathcal{T}_{j\to k}(e_i^k; S_i^{\text{pre}}, v_i^j, v_i^k)$$

In plain terms. This operator describes how mechanism j modifies mechanism k's effect.

be a directed modulation. The instance relation residual is defined *without* a graph-existence gate:

$$\boxed{r_i^{j\to k} = m_i^j m_i^k \left[\mathcal{T}_{j\to k}(e_i^k; S_i^{\text{pre}}, v_i^j, v_i^k) - e_i^k\right].}$$

In plain terms. This isolates the extra change caused by the directed relation $j \to k$.

This notation separates three facts: mechanisms participate in an experience through $m_i^j m_i^k$; the relation operator has an experience-specific realisation $r_i^{j\to k}$; and the candidate edge belongs to the learned domain structure according to a global gate g_{jk}.

The first relation network is a shared directed residual-FiLM network with a pair embedding $q_R^{j\to k}$. With $c_i = \text{AttnPool}(S_i^{\text{pre}})$,

$$(\gamma_i^{jk}, \beta_i^{jk}) = F_R(v_i^j, v_i^k, c_i, q_R^{j\to k}),$$

In plain terms. This predicts how strongly the relation should scale and shift the target effect.

$$\tilde{e}_{i,l}^{j\to k} = (1 + \gamma_i^{jk}) \odot e_{i,l}^k + \beta_i^{jk},$$

In plain terms. This applies the predicted scale and shift to mechanism k's effect.

and hence

$$r_i^{j\to k} = m_i^j m_i^k (\tilde{e}_i^{j\to k} - e_i^k).$$

In plain terms. This keeps only the relation-induced correction when both mechanisms are active.

Candidate relation existence is a domain-global structural parameter

$$\boxed{g_{jk} \sim \text{HardConcrete}(\alpha_{jk}).}$$

In plain terms. This learns whether the directed relation belongs to the world model at all.

The gate is applied only when the world aggregates relation realisations:

$$\boxed{\widehat{\Delta S}_i^W = \sum_j e_i^j + \sum_{j\neq k} g_{jk} r_i^{j\to k}.}$$

In plain terms. This predicts the total change from mechanism effects plus enabled relation corrections.

Equivalently,

$$\tilde{e}_i^k = e_i^k + \sum_{j\neq k} g_{jk} r_i^{j\to k}, \qquad \widehat{\Delta S}_i^W = \sum_k \tilde{e}_i^k.$$

In plain terms. This rewrites the same prediction by first updating each target effect, then summing all targets.

This corrected placement avoids defining the same structural gate both inside the relation realisation and again in the world graph. The world grounding loss is

$$\boxed{L_{\text{world}} = \sum_i D_S(\Delta S_i, \widehat{\Delta S}_i^W).}$$

In plain terms. This trains the full world model to match the observed state changes.

Real-world example. Arm motion and grasp closure each have baseline effects, but closure also determines how much arm motion transfers to the object. The directed relation captures this extra modulation, and its gate turns off if it adds no held-out value.

The first implementation represents pairwise effect modulation observable when both mechanisms participate. Activation of an otherwise absent target and stable higher-order interactions are outside this first-order model; they should be introduced only if sufficiently supported residuals persist.

Directional relation identifiability

Co-participation does not identify direction. Define the participation-aware source state

$$\boxed{\bar{z}_i^j = (m_i^j, m_i^j v_i^j).}$$

In plain terms. This records both participation and value so directional comparisons do not confuse absence with zero.

To support $j \to k$, experiences a, b should approximately match pre-state and target realisation while source j varies. Let

$$w_{ab}^{(k)} = m_a^k m_b^k K_S(S_a^{\text{pre}}, S_b^{\text{pre}}) K_v(v_a^k, v_b^k),$$

In plain terms. This gives high weight to experience pairs where the context and target mechanism are well matched.

$$d_{ab}^{(j)} = D_z(\bar{z}_a^j, \bar{z}_b^j).$$

In plain terms. This measures how much the proposed source mechanism changes across the matched pair.

Directional contrastive support is

$$\boxed{\mathcal{E}_{j\to k}^R = \Phi\left(\sum_{a<b} w_{ab}^{(k)} d_{ab}^{(j)}\right),}$$

In plain terms. This totals the available matched evidence for testing the direction $j \to k$.

where Φ is monotone and saturating. This is an evidence statistic, not relation strength, and generally

$$\mathcal{E}_{j\to k}^R \neq \mathcal{E}_{k\to j}^R.$$

In plain terms. This states that evidence for one direction does not automatically support the reverse direction.

Real-world example. If A and B always appear together, we cannot tell whether A changes B or B changes A. We need cases where B is held constant and A varies. In a manipulation experiment, this means holding the grasp condition fixed while changing arm motion and observing the object response; the reverse direction requires a separate contrast.

To test whether an edge has functional value, let $\widehat{\Delta S}_i^{-jk}$ remove $g_{jk} r_i^{j\to k}$ from the world prediction. The held-out explanatory power is

$$\boxed{EP_{j\to k} = \mathbb{E}_{i\sim\mathcal{D}_{\text{held}}}\left[D_S(\Delta S_i, \widehat{\Delta S}_i^{-jk}) - D_S(\Delta S_i, \widehat{\Delta S}_i^{W})\right].}$$

In plain terms. This measures how much held-out prediction worsens when relation $j \to k$ is removed.

An edge is unresolved when $\mathcal{E}^R_{j\to k}$ is insufficient; it is supported only when evidence is sufficient and $EP_{j\to k}$ remains positive across held-out contexts. Lack of evidence is not evidence of absence. Accordingly, let

$$s^R_{jk} = f(\mathcal{E}^R_{j\to k}),$$

In plain terms. This converts directional evidence into a confidence weight for judging the edge.

with $s^R_{jk} \approx 0$ under insufficient support. The support-conditioned edge penalty is

$$\boxed{L^{\text{support}}_{\text{edge}} = \sum_{j\neq k} s^R_{jk}\mathbb{E}[g_{jk}].}$$

In plain terms. This penalises unnecessary edges only when the data were sufficient to test them.

Only candidates that the data could have adjudicated pay a strong absence pressure.

Real-world example. If the robot never varied arm motion while maintaining grasp contact, it cannot conclude that arm motion does not affect transferred object motion. The edge penalty applies only when the data could have exposed that effect.

$Z_D \leftrightarrow R_D$ co-refinement and staged training

Let Σ^{int}_j summarise how interventions engage mechanism j, and define its relational signature

$$\mathcal{R}_j = \{R_D^{j\to k}, R_D^{k\to j}\}_{k\neq j}.$$

In plain terms. This collects all incoming and outgoing relations associated with mechanism j.

Operational identity is

$$\boxed{\text{Identity}(Z_D^j) = (\mathcal{M}_j, \Sigma^{\text{int}}_j, \mathcal{R}_j).}$$

In plain terms. This defines a mechanism by its effect, intervention pattern, and relational role together.

Relations refine variable identity but do not wholly define it. The overcomplete bank permits soft splitting when an inactive query acquires stable participation and soft deactivation when a redundant query's participation tends to zero.

Training proceeds in three phases:

1. **Candidate mechanisms.** Set $R_D = 0$ and optimise $L^{(0)}_{Z_D}$. Mechanisms must acquire baseline reusable roles before relation modules receive capacity.
2. **Relation residuals.** Freeze or strongly slow q_η and $\mathcal{M}_j$; enable F_R and g_{jk}; optimise

$$\boxed{L_R = L_{\text{world}} + \lambda_R L^{\text{support}}_{\text{edge}}.}$$

In plain terms. This trains relations to improve prediction while avoiding unsupported extra edges.

Relations must explain residual structure rather than replacing mechanism baselines.

3. **Joint co-refinement.** Unfreeze both levels at smaller learning rates. If structurally single-mechanism experiences exist,

$$\mathcal{D}_{\text{base}} = \{i : \|m_i\|_0 \approx 1\},$$

In plain terms. This selects experiences that approximately isolate one mechanism.

use

$$\boxed{L_{\text{base}} = \mathbb{E}_{i \in \mathcal{D}_{\text{base}}} D_S \left(\Delta S_i, \sum_j e_i^j \right).}$$

In plain terms. This keeps individual mechanism effects correct on isolated baseline experiences.

If such data do not exist, set $\lambda_B = 0$ rather than fabricating a baseline.

The co-refinement objective is

$$\boxed{L_{W_D} = L_{\text{world}} + \lambda_R L_{\text{edge}}^{\text{support}} + \lambda_P L_{\text{participation}} + \lambda_B L_{\text{base}} + \lambda_T L_{\text{TP-rec}}.}$$

In plain terms. This is the final joint loss for fitting the world while preserving sparsity, baselines, and TP consistency.

4.5 Active exploration: W_D back to interaction

Structural evidence, not a learned U_D

Uncertainty is not introduced as an independent latent U_D. The relevant question is how much evidence the current dataset contains for mechanism reuse and directed relation identification. Retain reliable Z_D experiences

$$\mathcal{D}_t^Z = \{x_i^Z\}_{i=1}^{N_t}, \qquad x_i^Z = (S_i^{\text{pre}}, m_i, v_i).$$

In plain terms. This stores the reliable mechanism-level experiences collected so far.

For mechanism j, define

$$\mathcal{D}_j = \{(S_i^{\text{pre}}, v_i^j) : m_i^j \approx 1\}.$$

In plain terms. This gathers the contexts and values in which mechanism j was active.

The first explicit evidence estimator uses

$$K_j[a, b] = m_a^j m_b^j K_S(S_a^{\text{pre}}, S_b^{\text{pre}}) K_v(v_a^j, v_b^j),$$

In plain terms. This builds a similarity matrix over the supported experiences of mechanism j.

$$\boxed{\mathcal{E}_j^Z = \log\det(I + \alpha_Z K_j).}$$

In plain terms. This scores how varied and non-redundant the evidence for mechanism j is.

Near-duplicate experiences add little; new supported contexts or realisations expand evidence. Relation evidence reuses $\mathcal{E}_{j\to k}^R$. The total statistic is

$$\boxed{\mathcal{E}_D(\mathcal{D}_t^Z; W_D) = \sum_j \mathcal{E}_j^Z + \lambda_E^R \sum_{j \neq k} \mathcal{E}_{j\to k}^R.}$$

In plain terms. This combines mechanism evidence and relation evidence into one domain-level score.

It is a functional of data and the current structural hypothesis, not a learned object.

Real-world example. Repeating the same push on the same block 100 times adds little structural evidence. Testing a new force, pose, friction, or constraint reveals more about whether the mechanism truly transfers.

Expected structural evidence gain

For a candidate intervention sequence A at state S_t,

$$p \sim p_\theta(p \mid S_t, A), \qquad (m, v) \sim q_\eta(m, v \mid p, S_t),$$

In plain terms. This simulates the transformation and mechanism engagement that a candidate action may produce.

which predicts $x_A^Z = (S_t, m, v)$. The expected evidence increment is

$$\boxed{J_{\text{evidence}}(A) = \mathbb{E}_{p,m,v}\left[\mathcal{E}_D(\mathcal{D}_t^Z \cup \{x_A^Z\}; W_D) - \mathcal{E}_D(\mathcal{D}_t^Z; W_D)\right].}$$

In plain terms. This estimates how much new structural evidence action A is expected to add.

This single term replaces separate learned variable-, relation-, pair-, and uncertainty policies. Exploration supplies evidence; ordinary $Z_D \leftrightarrow R_D$ learning decides which structure is needed.

TP competence and the abstraction frontier

Evidence completion alone confirms the current ontology. To search for omitted structure, compare the lower-level TP effect with the higher-level world explanation, but only where TP is locally reliable. Define

$$\boxed{\mathcal{C}_{TP} = \{(S, A) : \text{local prior–posterior error, TP effect error, and support are calibrated}\}.}$$

In plain terms. This marks the state–action regions where the TP model is reliable enough to guide exploration.

Historical estimates use

$$D_{TP}[q_\phi(p \mid S, S') \| p_\theta(p \mid S, A)], \qquad D_S(\Delta S, F_{TP}(S, p^{\text{post}})),$$

In plain terms. These errors check whether the TP was predicted correctly and whether it reproduces the observed change.

and neighbourhood support in (S, A). No new competence network is required initially.

For a predicted TP,

$$F_W(S, p) = \mathbb{E}_{q_\eta(m,v|p,S)}\left[\sum_j e^j + \sum_{j \neq k} g_{jk} r^{j \to k}\right],$$

In plain terms. This maps a predicted TP into the change expected by the current world model.

$$\boxed{F_D(S, p) = D_S(F_{TP}(S, p), F_W(S, p)).}$$

In plain terms. This measures disagreement between the lower-level TP prediction and the higher-level world explanation.

The prospective abstraction-frontier utility is

$$\boxed{J_{AF}(A) = \mathbb{E}_{p\sim p_\theta(p|S_t,A)}[F_D(S_t,p)], \quad (S_t,A) \in \mathcal{C}_{TP}.}$$

In plain terms. This favours reliable actions expected to expose gaps in the current world abstraction.

Real-world example. If the TP model reliably predicts a 20-cm slide but the mechanism model explains only 5 cm, the remaining gap suggests missing structure. Exploration then seeks similar interactions that can identify the missing contribution.

Large discrepancy does not prove missing structure; it proposes an experiment. After real execution, error attribution decides whether TP or W_D failed.

Minimal exploration objective and optimisation

The complete knowledge objective has two terms:

$$\boxed{J_{\text{exp}}(A \mid S_t) = J_{\text{evidence}}(A) + \lambda_F J_{AF}(A).}$$

In plain terms. This balances confirming uncertain structure with searching for missing structure.

Subject to physical, safety, and competence constraints,

$$\boxed{A_t^* = \underset{A\in\mathcal{A}_{\text{valid}}(S_t)\cap\mathcal{C}_{TP}(S_t)}{\arg\max} J_{\text{exp}}(A \mid S_t).}$$

In plain terms. This selects the safest reliable action with the greatest exploration value.

The reparameterised TP prior supplies $\partial p/\partial A$, and relaxed mechanism gates supply $\partial(m,v)/\partial p$. The first optimiser is projected gradient ascent with multiple safe initialisations:

$$\boxed{A^{(r+1)} = \Pi_{\mathcal{A}_{\text{valid}}\cap\mathcal{C}_{TP}}\left[A^{(r)} + \eta_A \nabla_A J_{\text{exp}}\right].}$$

In plain terms. This improves the action step by step, then projects it back into the valid region.

Discrete modes such as gripper open/close are enumerated; continuous components are optimised within each mode. Only a short prefix is executed before observation and replanning. A sampling planner is an evaluation fallback when gradients are unreliable, not a second exploration policy. Before any model is competent, safe broad interactions form $\mathcal{D}_{\text{seed}}$.

Post-interaction error attribution

After executing A, infer the posterior TP from the real outcome. Define

$$\boxed{E_{\text{prior}} = D_{TP}[q_\phi(p \mid S^{\text{pre}}, S^{\text{post}}) \| p_\theta(p \mid S^{\text{pre}}, A)],}$$

In plain terms. This checks whether the executed action produced the TP that the prior predicted.

$$\boxed{E_{TP} = D_S(\Delta S_{\text{real}}, F_{TP}(S^{\text{pre}}, p^{\text{post}})),}$$

In plain terms. This checks whether the inferred TP itself explains the real state change.

$$\boxed{E_W = D_S(\Delta S_{\text{real}}, F_W(S^{\text{pre}}, p^{\text{post}})), \qquad G_W^{\text{obs}} = E_W - E_{TP}.}$$

In plain terms. This checks the world abstraction and separates its extra error from TP-level error.

Large E_{prior} indicates intervention-to-TP failure; small E_{prior} but large E_{TP} indicates a TP representation/effect failure; small lower-level errors but large E_W indicate a world-abstraction failure. An experience can first update TP and later, once reliably represented, contribute to W_D.

4.6 Learning task constraint C_T from grouped demonstrations

TP sequence extraction

The frozen interaction and TP models segment every task demonstration:

$$\tau_{g,r} \longrightarrow P_{g,r} = (p_1^{g,r}, \ldots, p_{N_{g,r}}^{g,r}),$$

In plain terms. This converts each task demonstration into a sequence of learned transformations.

with associated boundary states $S_0^{g,r}, \ldots, S_{N_{g,r}}^{g,r}$. The first dataset supplies the task episode boundary and grouping g. It does not supply TP labels or a task code.

Task encoder

A causal or bidirectional sequence Transformer with positional encodings maps the complete support demonstration to

$$\boxed{c_{g,r} = E_C(P_{g,r}) \in \mathbb{R}^{d_C}.}$$

In plain terms. This compresses one demonstration's TP sequence into a task code.

When initial and terminal states carry information needed to disambiguate otherwise identical TP sequences, their set summaries may be included as tokens. This is an implementation interface; the scientific test is whether $c_{g,r}$ transfers functionally to other realisations of task g.

Explicit W_D interface

The task predictor does not receive only a monolithic state embedding. It receives an explicit world-structure readout containing the currently instantiated mechanism nodes and gated directed relations. For query step i,

$$(m_i, v_i) \sim q_\eta(m, v \mid p_i, S_i),$$

In plain terms. This infers the mechanisms active at the current task step.

$$\mathcal{W}_i = \left(\{q_D^j, m_i^j, v_i^j, e_i^j\}_j, \{g_{jk}, r_i^{j \to k}\}_{j \neq k} \right).$$

In plain terms. This gathers the active mechanisms and directed relations into an explicit world readout.

A one-layer set/graph encoder E_W produces query features while preserving node and directed-edge identity:

$$h_i^W = E_W(S_i, \mathcal{W}_i).$$

In plain terms. This encodes the current world structure for the task predictor.

This computation is an on-demand readout of W_D, not a persistent task graph G_T.

Multimodal task progression

Let

$$x_i^{g,s} = (S_i^{g,s}, h_i^{W,g,s}, p_i^{g,s}, c_{g,r})$$

In plain terms. This joins state, world, progress, and task information for next-step prediction.

for a query demonstration $s \neq r$. The next-TP distribution is a conditional Gaussian mixture:

$$\boxed{P_T(p_{i+1}^{g,s} \mid x_i^{g,s}) = \sum_{q=1}^{Q} \pi_q(x_i^{g,s}) \mathcal{N}\left(p_{i+1}^{g,s}; \mu_q(x_i^{g,s}), \mathrm{diag}(\sigma_q^2(x_i^{g,s}))\right).}$$

In plain terms. This predicts several possible next transformations instead of averaging them into one.

The start distribution omits a previous TP:

$$P_T(p_1^{g,s} \mid S_0^{g,s}, h_0^{W,g,s}, c_{g,r}),$$

In plain terms. This predicts the first transformation before any previous TP is available.

and a Bernoulli head predicts

$$P_T(y_i^{\text{stop}} \mid S_i^{g,s}, h_i^{W,g,s}, p_i^{g,s}, c_{g,r}).$$

In plain terms. This predicts whether the task should stop at the current step.

Multimodality prevents the mean of two valid strategies from becoming an invalid next TP and reduces pressure to encode strategy identity in C_T.

Real-world example. For placing a cup behind an obstacle, the next valid TP may move left or right around it. A mixture preserves both choices, while the stop head asks whether the cup has reached the target region.

Cross-demonstration objective

The support code from demonstration r must explain a different query demonstration s of the same task:

$$\boxed{L_{\text{start}}^{r \to s} = -\log P_T(p_1^{g,s} \mid S_0^{g,s}, h_0^{W,g,s}, c_{g,r}),}$$

In plain terms. This penalises an incorrect first-step prediction on another demonstration of the same task.

$$\boxed{L_{\text{next}}^{r \to s} = -\sum_{i=1}^{N_s - 1} \log P_T(p_{i+1}^{g,s} \mid S_i^{g,s}, h_i^{W,g,s}, p_i^{g,s}, c_{g,r}),}$$

In plain terms. This penalises incorrect next-TP predictions across the query demonstration.

$$\boxed{L_{\text{stop}}^{r \to s} = -\sum_{i=0}^{N_s} \log P_T(y_i^{\text{stop},g,s} \mid S_i^{g,s}, h_i^{W,g,s}, p_i^{g,s}, c_{g,r}).}$$

In plain terms. This penalises incorrect continue-or-stop decisions.

The task objective is

$$\boxed{L_{C_T} = \mathbb{E}_{g,r\neq s}\left[L_{\text{start}}^{r\to s} + L_{\text{next}}^{r\to s} + \lambda_{\text{stop}} L_{\text{stop}}^{r\to s}\right].}$$

In plain terms. This trains one demonstration's task code to predict a different demonstration of that task.

There is no required Euclidean consistency penalty $\|c_{g,r} - c_{g,s}\|$. Task codes are equivalent when they induce the same predictive function on held-out realisations. Self-reconstruction alone is not used because it admits a trajectory-identity solution.

Task state and task transformation

The minimal task state is

$$\boxed{Z_T = (S_t, C_T).}$$

In plain terms. This defines task state using only the current world state and task constraint.

The last TP p_t may be supplied to the next-TP head as a transient order-one progress cue, but it is not a new persistent component of Z_T. If an experiment shows that longer history is necessary, a progress memory can be added as a tested extension rather than assumed in advance.

Task transformation is the conditional operator

$$\boxed{\mathcal{T}_T(W_D, Z_T; p_t) = P_T(p_{\text{next}}, y^{\text{stop}} \mid S_t, W_D, C_T; p_t).}$$

In plain terms. This lets the task constraint select the next transformation from the explicit world model.

C_T acts on W_D by selecting, restricting, and reweighting its feasible transformation structure. The output remains a distribution. No independent D_T is needed to store one desired TP, and no explicit G_T is needed to store a task-specific subgraph before evidence shows that such a persistent object improves explanation or transfer.

C_T identifiability and necessity diagnostics

If (S_i, p_i, W_D) already predicts the query future, the optimum may ignore C_T. This is not necessarily optimisation failure; the dataset has not made task information necessary. Train a no-context baseline

$$P_0(p_{\text{next}}, y^{\text{stop}} \mid S, p, W_D)$$

In plain terms. This baseline predicts task progress without using the task code.

and report

$$\boxed{\Delta_{C_T} = \mathbb{E}_{\text{held}}\left[\log P_T(p_{\text{next}}, y^{\text{stop}} \mid S, p, W_D, C_T) - \log P_0(p_{\text{next}}, y^{\text{stop}} \mid S, p, W_D)\right].}$$

In plain terms. This measures whether the task code improves prediction on held-out data.

Positive held-out Δ_{C_T} under new initial states and demonstrations is evidence that C_T contributes information unavailable from the current world state. Strategy invariance is separately tested by transferring $c_{g,r}$ across demonstrations with different valid TP paths. World necessity is tested by comparing the full model against a predictor that removes W_D under the same task but different world configurations.

Real-world example. The same cup pose may require moving toward the tray in one task and toward the shelf in another. The task code earns its place only if it resolves such state-matched but goal-different cases.

4.7 Task action generation by TP-prior inversion

Distribution-level objective

At execution time, infer C_T from one or more support demonstrations, observe S_t, instantiate the explicit W_D readout, and obtain the desired distribution

$$P_T^*(p) = P_T(p_{\text{next}} = p \mid S_t, W_D, C_T; p_t).$$

In plain terms. This is the distribution of transformations currently desired by the task.

For candidate low-level intervention sequence A, the learned TP prior predicts

$$P_A(p) = p_\theta(p \mid S_t, A).$$

In plain terms. This is the distribution of transformations expected from candidate action A.

Action realisation is distribution matching:

$$\boxed{A_t^* = \underset{A \in \mathcal{A}_{\text{valid}} \cap \mathcal{C}_{TP}}{\arg\min} \{D_{TP}(P_A, P_T^*) + \lambda_a C_a(A)\}.}$$

In plain terms. This chooses a valid action whose predicted effect best matches the desired effect.

where C_a penalises unsafe, excessive, or unnecessarily long control. A Monte Carlo cross-entropy form that remains differentiable through p_θ is

$$\boxed{D_{TP}(P_A, P_T^*) = \mathbb{E}_{p \sim p_\theta(p|S_t,A)}[-\log P_T^*(p)].}$$

In plain terms. This scores an action by how probable its predicted effects are under the task objective.

To avoid exploiting an overly broad P_A, an entropy or variance penalty may be included as a controller regulariser. It is not a task-model loss.

Real-world example. If the desired TP is "slide the block moderately to the left," the controller searches for an action whose predicted effect matches that change. The action is obtained by inverting the TP prior rather than recalling a separate task policy.

First execution procedure

The first implementation uses the same projected multi-start gradient optimiser as exploration, but minimises the task objective. For highly separated TP modes, select or sample a valid mixture component q^* and minimise expected distance to its mean under component confidence. Enumerate discrete control modes, optimise continuous controls, execute a short prefix, observe, update S_t, and replan. Terminate when the stop posterior and safety checks agree.

No neural task motor policy π_ω^{task} and no extra supervised action loss are introduced. Exploration and task execution reuse the same learned map from intervention to transformation:

$$\boxed{\begin{array}{rcl} W_D & \to & J_{\text{exp}} \to A, \\ (W_D, S_t, C_T) & \to & P_T^*(p) \to p_\theta(p \mid S_t, A)^{-1} \to A. \end{array}}$$

In plain terms. This shows that exploration and task execution reuse the same action-to-transformation model.

This is the unifying control principle of the framework: higher levels assign value to transformations; the TP prior supplies their motor realisation.

4.8 Gradient boundaries and staged optimisation

The hierarchy is intended to abstract lower-level evidence, not rewrite it to fit a preferred high-level ontology. The first implementation therefore uses explicit stop-gradient boundaries.

During interaction and TP learning,

$$L_{\text{IR}}, L_{\text{TP}} \to E_{\text{vis}}, E_S, F_S, q_\phi, p_\theta, F_{TP}.$$

In plain terms. This lists the lower-level modules updated during interaction and TP learning.

During world abstraction,

$$\boxed{\text{stopgrad}(p), \qquad L_{W_D} \to q_\eta, \mathcal{M}_j, F_R, g_{jk},}$$

In plain terms. This trains the world abstraction while keeping the learned TP representation fixed.

but $L_{W_D} \nrightarrow q_\phi, p_\theta, E_S$. During task learning,

$$\boxed{\text{stopgrad}(TP), \quad \text{stopgrad}(W_D), \quad L_{C_T} \to E_C, P_T, E_W,}$$

In plain terms. This trains task reasoning without rewriting the TP or world models.

where E_W is only the task-side readout of the frozen explicit world structure. During exploration or task execution, all model parameters are frozen and gradients update only A.

Real-world example. If the mechanism layer could reshape the TP encoder, it might make every transformation easier for its current ontology to explain. Freezing the lower layer forces mechanisms to account for the transformations already supported by interaction data.

Stage	Optimised quantities	Frozen boundary
Interaction/TP	state encoder, dynamics, TP posterior/prior/effect, boundary	none between S and TP
Candidate Z_D	allocation and mechanism bank	TP representation
Relation discovery	relation network and edge gates	TP; initially mechanism bank
Co-refinement	Z_D and R_D at small learning rates	TP
Exploration	candidate intervention A	all model parameters
C_T learning	task encoder, multimodal predictor, W_D readout	TP and W_D
Task execution	candidate intervention A	all model parameters

High-level feedback still exists through data: W_D changes exploration, exploration changes the interaction distribution, and new reliable interactions update TP and W_D. This is system-level feedback rather than a direct gradient that could make lower evidence conform to the current ontology.

4.9 Unified objectives and closed loops

The training objectives are

$$\boxed{L_{IR} = L_{\text{dyn}},}$$

In plain terms. This restates the objective used to learn interaction representation.

$$\boxed{L_{TP} = L^{TP}_{\text{effect}} + \lambda_A L_{\text{align}} + \lambda_S L_{\text{seg}} + \lambda_C L_{\text{complexity}},}$$

In plain terms. This restates the full objective used to learn TPs.

$$\boxed{L_{Z_D}^{(0)} = L_{\text{effect}}^{D} + \lambda_P L_{\text{participation}} + \lambda_T L_{\text{TP}-\text{rec}},}$$

In plain terms. This restates the objective used to learn candidate mechanisms before relations.

$$\boxed{L_R = L_{\text{world}} + \lambda_R L_{\text{edge}}^{\text{support}},}$$

In plain terms. This restates the objective used to add supported relations.

$$\boxed{L_{W_D} = L_{\text{world}} + \lambda_R L_{\text{edge}}^{\text{support}} + \lambda_P L_{\text{participation}} + \lambda_B L_{\text{base}} + \lambda_T L_{\text{TP}-\text{rec}},}$$

In plain terms. This restates the joint objective used to refine mechanisms and relations together.

and the cross-demonstration L_{C_T} defined above. Exploration and task action selection are action-space objectives, not model-training losses.

The world-discovery loop is

$$\boxed{\textit{Interaction} \to TP \to Z_D \leftrightarrow R_D \to W_D \to \begin{cases} \textit{Evidence Completion} \\ \textit{Abstraction Challenge} \end{cases} \to \textit{Intervention} \to \textit{New Interaction}.}$$

In plain terms. This summarises how interaction improves the world model and guides the next interaction.

The task loop is

$$\boxed{\begin{aligned} &\textit{Grouped Demonstrations} \to TP\ \textit{Sequences} \to C_T, \\ &\qquad (S_t, C_T) + W_D \to P_T(p_{\text{next}}, \textit{stop}) \to TP\ \textit{Prior Inversion} \\ &\qquad\qquad \to A_t \to S_{t+1} \to \textit{Replan}. \end{aligned}}$$

In plain terms. This summarises how a task code becomes repeated transformation choices and actions.

Together they yield the full theory:

$$\boxed{\begin{aligned} &\textit{Interaction Representation} \to TP \to \begin{cases} Z_D \leftrightarrow R_D \to W_D \to \textit{exploration}, \\ TP\ \textit{sequences} \to C_T \end{cases} \\ &\qquad\qquad\qquad\qquad \to \textit{task transformation on } W_D \to \textit{action}. \end{aligned}}$$

In plain terms. This combines world discovery, task learning, exploration, and action into one hierarchy.

5. Computation and Experiments — Future Work

5.1 Status of this section

This section is a preregistered computational plan. It specifies the first implementation and the observations that would support or falsify each layer. It reports no completed experiment. Its purpose is to prevent later empirical convenience from silently redefining the theory.

"Preregistered" means: we write down what would count as success or failure before *running the experiment, so we cannot move the goalposts afterwards. This is the experimental equivalent of fixing an embodied-robotics evaluation protocol before collecting the test interactions.*

5.2 First-implementation network and algorithm specification

Component	First implementation	Reason for this first test
E_{vis}	ViT patch-token encoder	localised token interface for latent grouping
E_S	SAVi-like recurrent Slot Attention	persistent bounded-capacity set of interaction units
F_S	set Transformer with action FiLM	state-dependent intervention effects without concatenating action into state inference
correspondence	Hungarian matching	resolve residual training-time slot permutation
TP boundary	temporal Transformer with hazard head	variable duration and explicit termination probability
$q_\phi(p \mid S^{pre}, S^{post})$	Set Transformer plus TP query; diagonal Gaussian	permutation-aware multi-slot effect encoding and reparameterised baseline
$p_\theta(p \mid S^{pre}, A)$	state-set encoder plus action Transformer; diagonal Gaussian	prospective intervention-to-transformation map
F_{TP}	TP-conditioned set Transformer	distribute one TP effect across persistent interaction units
$q_\eta(m, v \mid p, S)$	mechanism-query cross-attention	simultaneous many-to-many allocation
m^j	Hard-Concrete participation gate	differentiable sparse local activation
v^j	diagonal-Gaussian realisation head	context-specific stochastic realisation
$\mathcal{M}_j$	shared set-dynamics backbone plus mechanism adapter	reuse generic dynamics while retaining mechanism identity
$R_D^{j\to k}$	shared directed residual-FiLM network plus pair embedding	constrain an edge to modify a target mechanism effect
g_{jk}	global Hard-Concrete edge gate	sparse domain-level candidate relation structure
world executor	one directed message-passing layer and explicit gated sum	preserve inspectable node/edge contribution
mechanism evidence	kernel log-determinant	explicit non-redundant coverage statistic
relation evidence	matched-context directional kernel support	measure whether source variation exists under comparable target context
exploration	projected multi-start gradient optimisation, receding horizon	use the differentiable TP bridge without a learned exploration policy
E_C	sequence Transformer over TP tokens	encode long-horizon persistent task constraint
P_T	task/world-conditioned Gaussian mixture plus stop head	represent multiple valid next transformations
task control	TP-prior distribution matching, projected multi-start optimisation, receding horizon	reuse intervention competence without a separate task motor policy

These choices define the first falsifiable system. If gradients are numerically unreliable, a sampling optimiser is a controller ablation; if a diagonal-Gaussian TP cannot represent observed transformations, the distribution may be enriched. Such changes test implementation adequacy and do not alter the physical priors unless failure persists across credible realisations.

5.3 Training-data design implied by the theory

World data

The unit of collection is the continuous, time-aligned trajectory, not an isolated image/action pair and not a video pre-cut at semantic action boundaries. The first dataset is organised as

$$\boxed{\mathcal{D}_{\text{world}} = \mathcal{D}_{\text{broad}} \cup \mathcal{D}_{\text{contrast}} \cup \mathcal{D}_{\text{explore}}.}$$

- $\mathcal{D}_{\text{broad}}$ uses safe low-level coverage to bootstrap S and TP.
- $\mathcal{D}_{\text{contrast}}$ varies initial conditions and controllable physical configurations to expose diverse mechanism combinations and directional contrasts. Environment design is not latent supervision: the model still receives only observations and executed interventions.

- $\mathcal{D}_{\text{explore}}$ is collected by the learned evidence/frontier objective after TP calibration.

Five coverage axes matter more than raw sample count:

1. **State coverage:** comparable interventions in different S.
2. **Intervention coverage:** different A from comparable S.
3. **Mechanism-combination coverage:** reusable mechanisms occur alone when possible and in varied combinations rather than always co-varying.
4. **Realisation coverage:** each active mechanism spans multiple v^j values and contexts.
5. **Directional contrast coverage:** target k and its context are approximately controlled while source j varies, and separate data support the reverse direction.

What matters is not "more data" but "data with the right contrasts": the same action in different states, different actions in the same state, each mechanism seen alone and in combinations, each mechanism seen at many strengths, and each relation tested in both directions.

The first proof-of-concept should use a simulator in which the experimenter knows, but does not train on, controllable mechanisms such as contact, friction, support, containment, attachment, collision, blocking, and grasp-mediated motion. Ground truth then evaluates whether learned variables are reusable and structurally aligned without supplying their names.

Task data

Every task group must contain multiple demonstrations. For the first experiment, $R_g = 5$–10 is a practical target, although the objective only requires $R_g \geq 2$. Within a task group, demonstrations deliberately vary:

- initial state;
- valid strategy and TP path;
- world configuration, including constraints and obstacles;
- low-level execution details such as speed and precise motor trajectory.

Across task groups, the dataset must contain locally similar states and TPs with different valid futures:

$$\boxed{(S_i^g, p_i^g, W_D) \approx (S_j^h, p_j^h, W_D),\ g \neq h, \quad P(p_{\text{next}} \mid C_T^g) \neq P(p_{\text{next}} \mid C_T^h).}$$

For the task code to be learnable, there must be situations that look the same in the world but belong to different tasks, so the only thing that decides "what next" is the task itself.

Real-world example. The same cup may begin at the same pose, while one task requires placing it in a tray and another requires leaving it clear of the tray. The world state alone cannot decide the next TP; the task code must resolve the difference.

Otherwise C_T is not identifiable as a necessary predictor. Within one task, the data must also contain different world configurations requiring different valid next TPs:

$$\boxed{C_T \text{ fixed}, \quad (S, W_D) \text{ varies} \quad \Rightarrow \quad P_T(p_{\text{next}}) \text{ varies}.}$$

Otherwise the task predictor can memorise a fixed script and ignore W_D. Cases with the same start and end but different middle processes test $C_T \neq$ trajectory; cases where the start and end are similar but a specific intermediate transformation is required test $C_T \neq$ terminal state.

Finally, task TP support must overlap the action-realisable world TP support:

$$\boxed{\text{supp}(TP_{\text{task}}) \subseteq \text{supp}_{\text{competent}}(p_\theta(p \mid S, A)) \text{ approximately}.}$$

This separates failure of task inference from failure to realise a correctly requested TP.

The transformations the task demands must be ones the world model can actually produce with some action. Otherwise we cannot tell "the task was understood wrong" from "the task was understood right but could not be performed."

5.4 Staged computational protocol

1. Collect $\mathcal{D}_{\text{seed}}$ with broad safe low-level controls and strict sensor/action synchronisation.
2. Train E_{vis}, E_S, F_S and validate temporal state correspondence under held-out trajectories.
3. Train boundary inference, q_ϕ, p_θ, and F_{TP}; validate both action/state contrasts and action realisability.
4. Freeze TP; train candidate Z_D without relations; select weak sparsity by held-out mechanism reuse, not by the prettiest decomposition.
5. Freeze or slow Z_D; train directional relation residuals only where contrast support is nontrivial.
6. Jointly co-refine $Z_D \leftrightarrow R_D$ at small learning rates; report gauge sensitivity across seeds.
7. Activate evidence/frontier exploration; interleave short collection batches with recalibration and staged retraining.
8. Freeze TP and W_D; segment grouped task demonstrations and train E_C, P_T with support/query pairs $r \neq s$.
9. Evaluate C_T and W_D necessity before attempting closed-loop control.
10. Execute task control by TP-prior inversion with receding-horizon replanning and error attribution.

This is the recipe, in order: gather data → learn state → learn transformations → learn mechanisms → learn relations → refine both → explore actively → learn the task → verify the task and world are necessary → act. Each step is validated before moving on.

5.5 Identifiability, necessity, and falsification diagnostics

Claim	Required diagnostic	Evidence against the claim
Persistent S	matching stability, held-out action-conditioned transition error, slot perturbation consistency	arbitrary rematching or equal performance from unstable framewise latents
TP is transformation, not action	same-action/different-state and different-action/similar-effect retrieval; prior–posterior calibration	TP predicted from action alone or unable to merge equivalent effects
TP boundary is meaningful	within-segment effect sufficiency and boundary perturbation test	boundaries coincide mechanically with action switches or every timestep
Z_D is reusable	cross-context and held-out-combination prediction by fixed $\mathcal{M}_j$	query-specific memorisation, universal participation, or no compositional transfer
R_D is directed structure	$\mathcal{E}^R_{j\to k}$, edge removal $EP_{j\to k}$, reverse-direction comparison	edge survives without contrast, has no held-out gain, or flips across seeds/contexts
W_D adds abstraction	world prediction relative to TP decoder, held-out recombination, description length	monolithic TP decoder matches all performance and structure is unstable
exploration is structural	evidence gained per interaction, edge/variable resolution rate, frontier discoveries	novelty/random coverage matches or beats the objective at equal budget
C_T is necessary	held-out Δ_{C_T} on locally matched cross-task states	no-context predictor performs equally
C_T is strategy-invariant	cross-strategy $r \to s$ likelihood and task retrieval by functional transfer	code follows trajectory/demonstrator rather than task group
C_T acts on W_D	same-task/different-world next-TP prediction; remove-W_D ablation	fixed script succeeds equally or W_D removal has no cost
Task TP is realisable	distribution match before/after action, execution success within TP competence	correct next-TP prediction but systematic inversion failure
No D_T/G_T needed	compare minimal operator with explicit-latent/graph ablations only after minimal failure	persistent residual requiring stable desired state or task graph

For every claim the framework makes, this table states the experiment that would confirm it and the result that would refute it. For example, "the TP is a transformation, not an action code" is confirmed if the same action in different states gives different TPs and different actions giving the same effect give the same TP; it is refuted if the TP can be predicted from the action alone.

Latent alignment metrics against simulator ground truth are secondary. Because representations have permutation and invertible-coordinate freedoms, the primary measurements are intervention prediction, held-out recombination, cross-context reuse, relation removal, cross-demonstration prediction, and control consequences.

5.6 Core ablations

The first study should isolate scientific necessity rather than enumerate every architecture. The mandatory ablations are:

1. action-conditioned versus action-free latent dynamics;
2. posterior without action versus posterior with action, to expose action-code leakage;
3. fixed one-step TP versus learned variable duration;
4. one-to-one TP clustering versus many-to-many mechanism allocation;
5. mechanisms only versus $Z_D + R_D$;
6. naive edge sparsity versus support-conditioned edge sparsity;
7. random/novelty exploration versus evidence only versus evidence plus abstraction frontier;
8. self-demonstration task reconstruction versus cross-demonstration prediction;
9. unimodal versus mixture next-TP prediction;
10. no C_T, full C_T, and shuffled task grouping;
11. no W_D, frozen explicit W_D, and a capacity-matched monolithic world embedding;
12. direct behaviour cloning versus TP-prior inversion at equal task data budget.

An "ablation" is the scientific version of "remove one part and see if it was doing anything." Each line removes or swaps one piece and asks whether the result gets worse. This isolates which pieces are necessary *rather than decorative.*

5.7 Failure modes to monitor

- **Slot gauge failure:** temporally unstable interaction units make change meaningless.
- **TP leakage:** the posterior memorises the post-state or the prior becomes an action embedding.
- **Mechanism collapse:** all m^j activate or one mechanism explains all effects.
- **Mechanism/relation gauge exchange:** $\mathcal{M}_j$ and R_D trade explanatory mass without stable held-out identity.
- **False edge absence:** sparse penalties prune an edge before directional evidence exists.
- **Ontology confirmation:** evidence-only exploration never proposes transformations outside the current Z_D support.
- **Model exploitation:** action optimisation finds adversarial controls outside calibrated TP support.
- **Task-code shortcut:** C_T encodes initial state, strategy, trajectory length, or demonstrator identity.
- **World bypass:** P_T predicts a memorised task script and ignores explicit W_D.
- **Control confounding:** task inference is blamed for an unrealisable TP or a poor local optimiser.
- **Partial observability:** hidden state makes identical observed S respond differently; longer memory or additional sensing is then required.

These are the ways the project is most likely to fail, written down in advance so that a failure can be diagnosed *rather than just noticed. Each failure has a known signature and a known fix.*

6. Preliminary Experiments: Learning Reusable Mechanisms from Local Change

6.1 Hypothesis and scope before E0

E0 isolates the first unresolved step between TP and Z_D: whether short-horizon changes can be explained by a small set of reusable mechanisms. Before introducing relations, the working model is

$$\boxed{\Delta S_i = \sum_j m_i^j \mathcal{M}_j(S_i, v_i^j).}$$

In plain terms. A TP is represented as the combined effect of the mechanisms active in that experience.

Here $\mathcal{M}_j$ is a mechanism function reused across TPs, m_i^j records whether it participates, and v_i^j describes its current realisation. The central assumption is not that visually similar TPs form one cluster. One mechanism may look different across states and intervention strengths, while one TP may contain several mechanisms. The stronger and more useful hypothesis is

Realisations may vary, but the response law of one mechanism should remain reusable across TPs.

In plain terms. Mechanism identity is tied to a shared response law rather than one observed effect.

E0 uses a synthetic world with $R_D = 0$ to remove relational ambiguity. It therefore does not attempt to recover complete Z_D identity. Its narrower question is whether mechanism-intrinsic evidence alone can produce stable reusable mechanism functions.

6.2 What E0 establishes

The mechanism-composition representation is expressive enough

In controlled tests, the model learns the mechanisms and recombines them to predict unseen TPs when participation or the correct mechanism structure is supplied. This separates representational adequacy from discovery: the proposed mechanism bank can express the desired solution, and the main difficulty lies in finding that solution without structural labels.

The compositional mechanism model is viable; unsupervised allocation is the bottleneck.

In plain terms. Oracle structure succeeds, so later failures cannot be blamed only on model expressivity.

Accurate prediction does not imply structural recovery

When the model must infer participation, mechanism functions, and mechanism count jointly from complete TPs, low prediction error does not reliably recover the generating structure. Several candidates can divide the effect of one true mechanism and produce an incorrect but self-consistent decomposition. Prediction and local sparsity alone therefore leave many equivalent explanations.

Predicting the change correctly is not the same as discovering the correct world structure.

In plain terms. A model may fit every TP while assigning its effects to the wrong mechanisms.

This result rejects the implicit learning assumption that end-to-end TP reconstruction plus sparse participation will automatically create correct mechanism identities.

Mechanisms require response families and a shared realisation coordinate

Single effects provide too little information about identity. In early runs, the model frequently split one true mechanism by response direction, magnitude, or state region. More reliable structure emerged when a controlled local intervention was varied across several strengths and the resulting changes were treated as a *response family*: a set of comparable observations of how one local effect changes with intervention.

Grouping the responses was not sufficient by itself. The model also needed their relative location along a shared variation axis — for example, which intervention was stronger and how the response changed with that strength. A realisation coordinate derived from the executed intervention supplied this ordering. With both response-family support and the shared coordinate, mechanism functions became substantially more stable across states and intervention strengths.

> A mechanism is characterised by a reusable response law,
> not by any single realised effect.

In plain terms. Identity becomes clearer when multiple responses reveal one common law of variation.

Mechanism count becomes easier after complete functions form

Once candidates have learned complete functions, redundant copies can be tested directly: evaluate whether they remain functionally interchangeable across states, realisation values, and held-out conditions. Candidates that make the same predictions throughout these tests can be merged or pruned. In E0 this functional comparison reduced an initial bank of five candidates toward the required count without providing that count during training.

> Counting mechanisms is not the main difficulty;
> forming complete and stable mechanisms comes first.

In plain terms. Redundant candidates are easy to remove only after each candidate represents a full function.

From-zero formation remains unstable

From-zero runs still sometimes divide one true mechanism among local candidates. The diagnostics show that these extra candidates do not all have the same cause. In the C4/C5 case, the original mechanism network could not cover two state regions at once. Adding a small mechanism-specific branch allowed C4 to absorb C5, while incorrect candidates could not do so; this failure was therefore consistent with insufficient function capacity.

The C2/C3 case remained unresolved. Changing the internal coordinate, retraining a unified mechanism, allowing two internal coordinates, and modifying the shared representation did not yield a stable merger. At the same time, C2 did not reveal a clear state region that only it could explain. The current evidence therefore supports neither the claim that C2 is a necessary mechanism nor the claim that it is merely a capacity-induced duplicate.

> Mechanism-intrinsic evidence in E0 does not yet guarantee
> a unique and complete structure from random initialisation.

In plain terms. The present method improves stability but does not eliminate every identity ambiguity.

6.3 How E0 changes the learning hypothesis

E0 does not reject the physical/structural prior that TPs admit composition by reusable mechanisms:

A small set of cross-experience mechanisms can compose observed TPs.

In plain terms. The basic world representation remains consistent with the controlled evidence.

It rejects a learning shortcut: jointly fitting complete TPs with several sparse candidates is not enough to make the candidates self-organise into the correct Z_D. Mechanism discovery is now treated as staged structure formation:

$$\begin{aligned} &\textit{Local intervention variation} \rightarrow \textit{comparable response family} \rightarrow \textit{shared realisation coordinate} \\ &\qquad \rightarrow \textit{provisional mechanism} \rightarrow \textit{cross-condition reuse test} \\ &\qquad \rightarrow \textit{functional redundancy removal} \rightarrow \textit{full-TP composition}. \end{aligned}$$

In plain terms. The revised procedure forms and tests mechanisms before using them to decompose complete TPs.

This revision has five operational consequences:

1. Begin with controlled local interventions rather than immediately allocating every complete TP.
2. Identify a mechanism from a family of responses across states and strengths, not from one effect.
3. Require different realisations of one mechanism to share a common variation rule.
4. Retain a candidate only when it contributes independent held-out predictive value.
5. Merge candidates only after broad functional interchangeability has been demonstrated.

The resulting position is

Mechanism discovery is not data clustering; it is the construction of response laws reusable across interventions and contexts.

In plain terms. The revised learner searches for transferable functions rather than groups of similar effects.

6.4 Boundary of the current evidence and next experiment

E0 studies mechanism-intrinsic structure under $R_D = 0$. It supports response families, a shared realisation coordinate, and function-based model selection as useful components, but it does not establish unique full mechanism identity. The strongest warranted limitation is therefore

Self-effect evidence alone cannot always remove ambiguity in mechanism identity.

In plain terms. Some candidates remain indistinguishable when only their own effects are considered.

The next experiment returns to the full theory by introducing $R_D \neq 0$. It will construct pairs of mechanisms whose intrinsic effects are deliberately similar but whose interactions with other mechanisms differ. The decisive comparison is between a learner using only intrinsic response families and one that also uses stable relational signatures.

This experiment will test — rather than assume — the stronger identity claim

An effective variable is defined jointly by what it does and by its stable relational position in the world.

In plain terms. Relations should distinguish mechanisms that cannot be separated by self-effects alone.

A positive result would show that $Z_D \leftrightarrow R_D$ co-refinement resolves ambiguities left by E0. A negative result would localise the remaining problem more sharply: relational context would then be insufficient, requiring either stronger intervention coverage or a revised definition of mechanism identity. No claim about this relational resolution is made by E0 itself.

References

[1] Locatello, F., Weissenborn, D., Unterthiner, T., Mahendran, A., Heigold, G., Uszkoreit, J., Dosovitskiy, A., & Kipf, T. **Object-Centric Learning with Slot Attention.** NeurIPS, 2020.

[2] Kipf, T., Elsayed, G. F., Mahendran, A., Stone, A., Sabour, S., Heigold, G., Jonschkowski, R., Dosovitskiy, A., & Greff, K. **Conditional Object-Centric Learning from Video.** ICLR, 2022.

[3] Mosbach, M., Ewertz, J. N., Villar-Corrales, A., & Behnke, S. **SOLD: Slot Object-Centric Latent Dynamics Models for Relational Manipulation Learning from Pixels.** ICML, 2025.

[4] Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houlsby, N. **An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale.** ICLR, 2021.

[5] Sutton, R. S., Precup, D., & Singh, S. **Between MDPs and Semi-MDPs: A Framework for Temporal Abstraction in Reinforcement Learning.** Artificial Intelligence, 112(1–2):181–211, 1999.

[6] Shankar, T., & Gupta, A. **Learning Robot Skills with Temporal Variational Inference.** ICML, 2020.

[7] Ghaemi, H., Muller, E. B., & Bakhtiari, S. **seq-JEPA: Autoregressive Predictive Learning of Invariant-Equivariant World Models.** NeurIPS, 2025.

[8] Nikulin, A., Zisman, I., Tarasov, D., Lyubaykin, N., Polubarov, A., Kiselev, I., & Kurenkov, V. **Latent Action Learning Requires Supervision in the Presence of Distractors.** ICML, 2025.

[9] Chen, G., Shao, Q., Cui, T., Zhou, Z., Mao, W., Yang, L., Wang, M., Yang, Y., Chen, H., & Yue, Y. **Learning a Unified Latent Action Space from Videos with Action-centric Cycle Consistency.** CVPR, 2026.

[10] Lippe, P., Magliacane, S., Löwe, S., Asano, Y. M., Cohen, T., & Gavves, E. **CITRIS: Causal Identifiability from Temporal Intervened Sequences.** ICML, 2022.

[11] Lippe, P., Magliacane, S., Löwe, S., Asano, Y. M., Cohen, T., & Gavves, E. **BISCUIT: Causal Representation Learning from Binary Interactions.** UAI, 2023.

[12] Lachapelle, S., Rodriguez, P., Sharma, Y., Everett, K. E., Le Priol, R., Lacoste, A., & Lacoste-Julien, S. **Disentanglement via Mechanism Sparsity Regularization: A New Principle for Nonlinear ICA.** Conference on Causal Learning and Reasoning, 2022.

[13] Bing, S., Ninad, U., Wahl, J., & Runge, J. **Identifying Linearly-Mixed Causal Representations from Multi-Node Interventions.** Conference on Causal Learning and Reasoning, 2024.

[14] Rajendran, G., Reizinger, P., Brendel, W., & Ravikumar, P. K. **An Interventional Perspective on Identifiability in Gaussian LTI Systems with Independent Component Analysis.** Conference on Causal Learning and Reasoning, 2024.

[15] Ahuja, K., Mahajan, D., Wang, Y., & Bengio, Y. **Interventional Causal Representation Learning.** ICML, 2023.

[16] Lippe, P., Magliacane, S., Löwe, S., Asano, Y. M., Cohen, T., & Gavves, E. **Causal Representation Learning for Instantaneous and Temporal Effects in Interactive Systems.** ICLR, 2023.

[17] Griffiths, T. L., & Ghahramani, Z. **Infinite Latent Feature Models and the Indian Buffet Process.** NeurIPS, 2005.

[18] Lee, J., Lee, Y., Kim, J., Kosiorek, A., Choi, S., & Teh, Y. W. **Set Transformer: A Framework for Attention-based Permutation-Invariant Neural Networks.** ICML, 2019.

[19] Zaheer, M., Kottur, S., Ravanbakhsh, S., Póczos, B., Salakhutdinov, R., & Smola, A. J. **Deep Sets.** NeurIPS, 2017.

[20] Louizos, C., Welling, M., & Kingma, D. P. **Learning Sparse Neural Networks through L_0 Regularization.** ICLR, 2018.

[21] Perez, E., Strub, F., de Vries, H., Dumoulin, V., & Courville, A. **FiLM: Visual Reasoning with a General Conditioning Layer.** AAAI, 2018.

[22] Battaglia, P. W., Hamrick, J. B., Bapst, V., et al. **Relational Inductive Biases, Deep Learning, and Graph Networks.** arXiv:1806.01261, 2018.

[23] Lindley, D. V. **On a Measure of the Information Provided by an Experiment.** The Annals of Mathematical Statistics, 27(4):986–1005, 1956.

[24] Krause, A., Singh, A., & Guestrin, C. **Near-Optimal Sensor Placements in Gaussian Processes: Theory, Efficient Algorithms and Empirical Studies.** Journal of Machine Learning Research, 9:235–284, 2008.

[25] Pathak, D., Agrawal, P., Efros, A. A., & Darrell, T. **Curiosity-driven Exploration by Self-supervised Prediction.** ICML, 2017.

[26] Amos, B., Jimenez Rodriguez, I. D., Sacks, J., Boots, B., & Kolter, J. Z. **Differentiable MPC for End-to-end Planning and Control.** NeurIPS, 2018.

[27] Chua, K., Calandra, R., McAllister, R., & Levine, S. **Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models.** NeurIPS, 2018.

[28] Ebert, F., Finn, C., Dasari, S., Xie, A., Lee, A. X., & Levine, S. **Visual Foresight: Model-Based Deep Reinforcement Learning for Vision-Based Robotic Control.** arXiv:1812.00568, 2018.

[29] Garrido, Q., Najman, L., & LeCun, Y. **Self-supervised Learning of Split Invariant Equivariant Representations.** ICML, 2023.

[30] Freed, B., James, S., Sartoretti, G., & Choset, H. **Learning Temporally Abstract World Models without Online Experimentation.** ICML, 2023.

[31] Mishra, N., Abbeel, P., & Mordatch, I. **Prediction and Control with Temporal Segment Models.** ICML, 2017.

[32] Locatello, F., Bauer, S., Lucic, M., Raetsch, G., Gelly, S., Schölkopf, B., & Bachem, O. **Challenging Common Assumptions in the Unsupervised Learning of Disentangled Representations.** ICML, 2019.

[33] Khemakhem, I., Kingma, D. P., Monti, R. P., & Hyvärinen, A. **Variational Autoencoders and Nonlinear ICA: A Unifying Framework.** AISTATS, 2020.

[34] Finn, C., Yu, T., Zhang, T., Abbeel, P., & Levine, S. **One-Shot Visual Imitation Learning via Meta-Learning.** Conference on Robot Learning, 2017.

[35] James, S., Bloesch, M., & Davison, A. J. **Task-Embedded Control Networks for Few-Shot Imitation Learning.** Conference on Robot Learning, 2018.

[36] Rakelly, K., Zhou, A., Finn, C., Levine, S., & Quillen, D. **Efficient Off-Policy Meta-Reinforcement Learning via Probabilistic Context Variables.** ICML, 2019.

[37] Garnelo, M., Rosenbaum, D., Maddison, C. J., Ramalho, T., Saxton, D., Shanahan, M., Teh, Y. W., Rezende, D. J., & Eslami, S. M. A. **Conditional Neural Processes.** ICML, 2018.

[38] Bishop, C. M. **Mixture Density Networks.** Aston University Technical Report NCRG/94/004, 1994.

[39] Chi, C., Feng, S., Du, Y., Xu, Z., Cousineau, E., Burchfiel, B., & Song, S. **Diffusion Policy: Visuomotor Policy Learning via Action Diffusion.** Robotics: Science and Systems, 2023.

[40] Srinivas, A., Jabri, A., Abbeel, P., Levine, S., & Finn, C. **Universal Planning Networks: Learning Generalizable Representations for Visuomotor Control.** ICML, 2018.